Implementation of Multi-Agent Reinforcement Learning for redundant robot control in task-space

Kothawale Chetan Vishwanath R

Indian Institute of Technology Jodhpur

07/04/2025

Table of Contents

- Introduction
- Moore-Penrose pseudo inverse based Jacobian
- Singularity-Avoidance based Jacobian
- Q-Learning based Multi-Agent RL method
- Conclusion

Introduction

- The paper [1] focuses on the challenges faced in the task-space control of a robot.
- The authors carried out this research considering an example of 4-DoF manipulator arm.
- They have compared their proposed actor-critic based Multi-Agent Reinforcement Learning (MARL) method with traditional Jacobian methods and Neural Network methods.
- Their results have proven that their MARL methods is much more better compared with the classical methods.

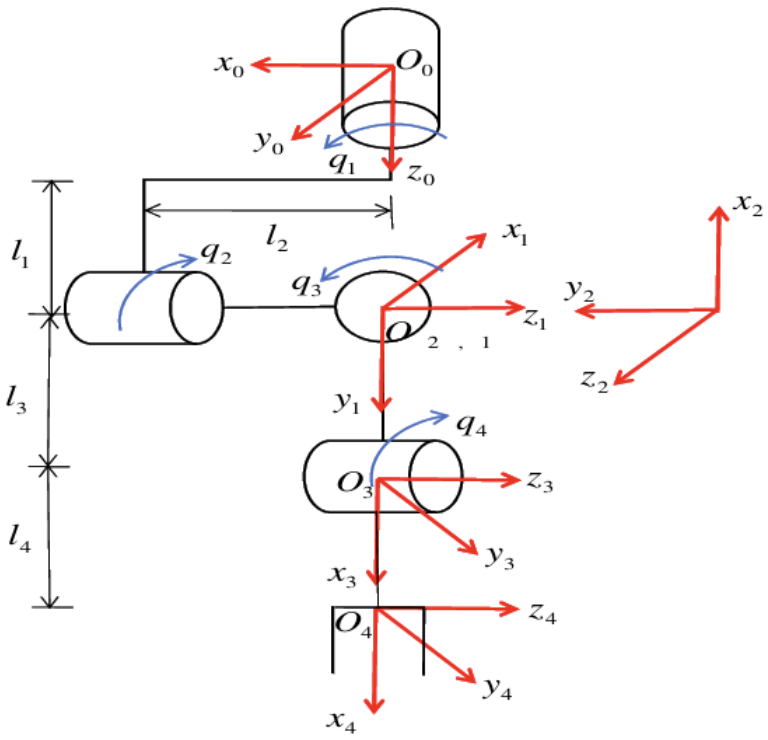
[1]

[1] [1] A. Perrusquia, W. Yu, and X. Li, "Multi-agent reinforcement learning for redundant robot control in task-space," International Journal of Machine Learning and Cybernetics, vol. 12, no. 1, pp. 231–241, 2020. [Online]. Available: https://doi.org/10.1007/s13042-020-01167-7

Introduction (Cont.)

- After reading the manuscript, we have understood and analysed the methods implemented by the authors.
- We have understood that their whole research methods were verified with the 4-DoF arm the authors had.
- Since we do not have the same setup and we did find a bit impractical to find an arm with the given D-H frame.
- We have explored the classical Jacobian methods such as Moore-Penrose pseudo inverse method, and Jacobian-based Singular Avoidance (J+SA) and Q-Learning based MARL method assuming each joint as an individual agent.

Introduction (Cont.)



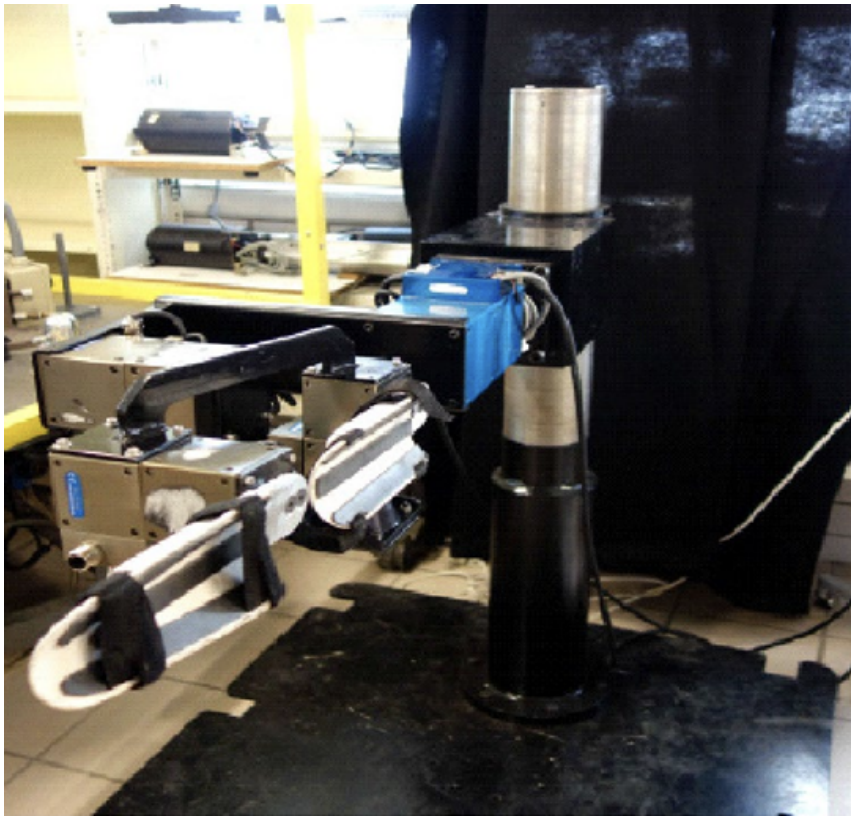


Figure: Robot configuration as presented by the authors

Overview of Methods

1
Moore-Penrose pseudo inverse based Jacobian method

2
Singular Avoidance based Jacobian method

3
Q-Learning based Multi-Agent RL method

Implementation of Jacobian based methods

- Computing the Jacobian matrix ensures accurate trajectory tracking, but computing this analytical Jacobian method is difficult.
- We use numerical techniques that can approximate the Jacobian. We assume the perturbation δ to be infinitesimally small such that it is as good as computing for every single point in the trajectory.
- Given a small perturbation δ in each joint variable q_i, the i-th column of the Jacobian J is estimated as:

$$J(:, i) \approx \frac{f(q + \delta e_i) - f(q)}{\delta}$$

  where $f(q)$ is the forward kinematics function and e_i is a unit vector with 1 at the i-th position.
- With this assumption, we carry forward to perform both the proposed Jacobian methods in simulation.

Moore-Penrose pseudo inverse based Jacobian

- The 4-DoF manipulator arm has 4 joint space variables ($n = 4$) and 3 task space variables ($m = 3$), resulting in a non-square Jacobian matrix ($J \in \mathbb{R}^{3\times4}$).
- To find the pseudo-inverse of the Jacobian matrix we use the Moore-Penrose method, given by:

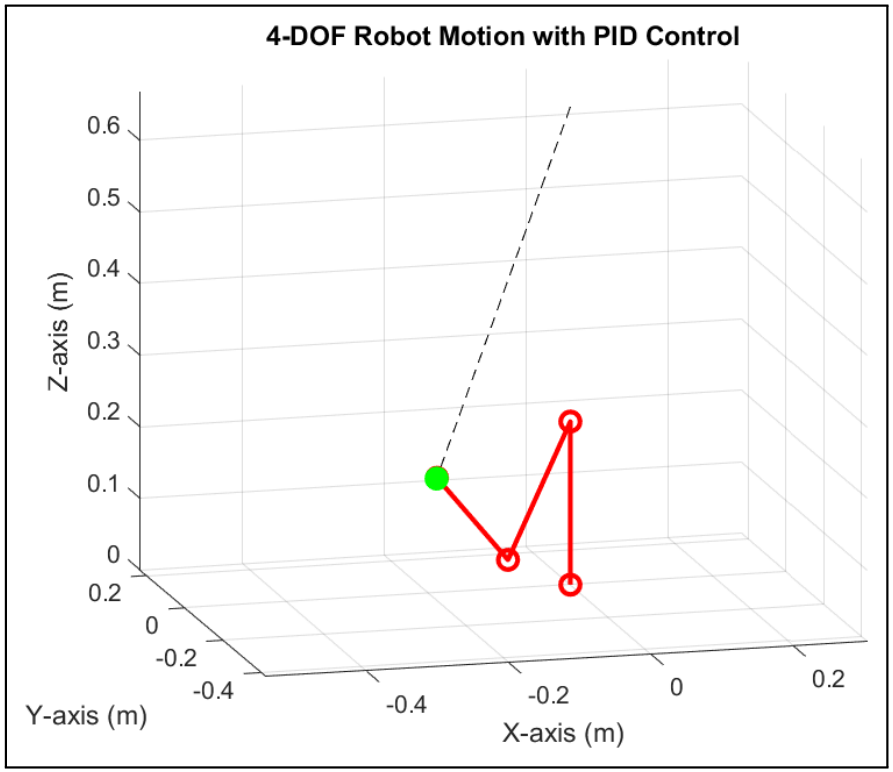
$$J^+ = J^T(JJ^T)^{-1}$$

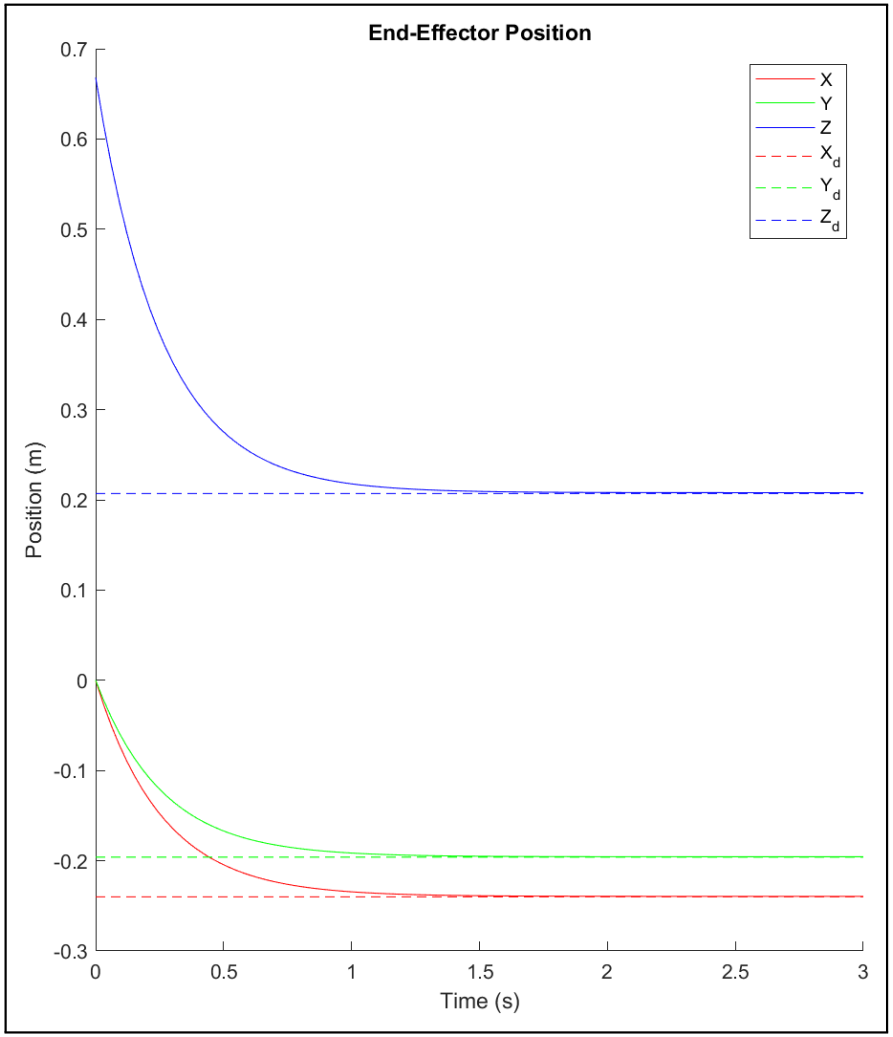
  This provides the minimum norm solution for joint velocities $\dot{q} = J^+\dot{x}$.
- To ensure constant control of the trajectory, we implement PID control law for the task space error $e = x_d - x$:

$$u = K_p e + K_i \int e dt + K_d \frac{de}{dt}$$

  (Where u represents the desired task-space velocity or force).

Moore-Penrose Jacobian: Simulation Results





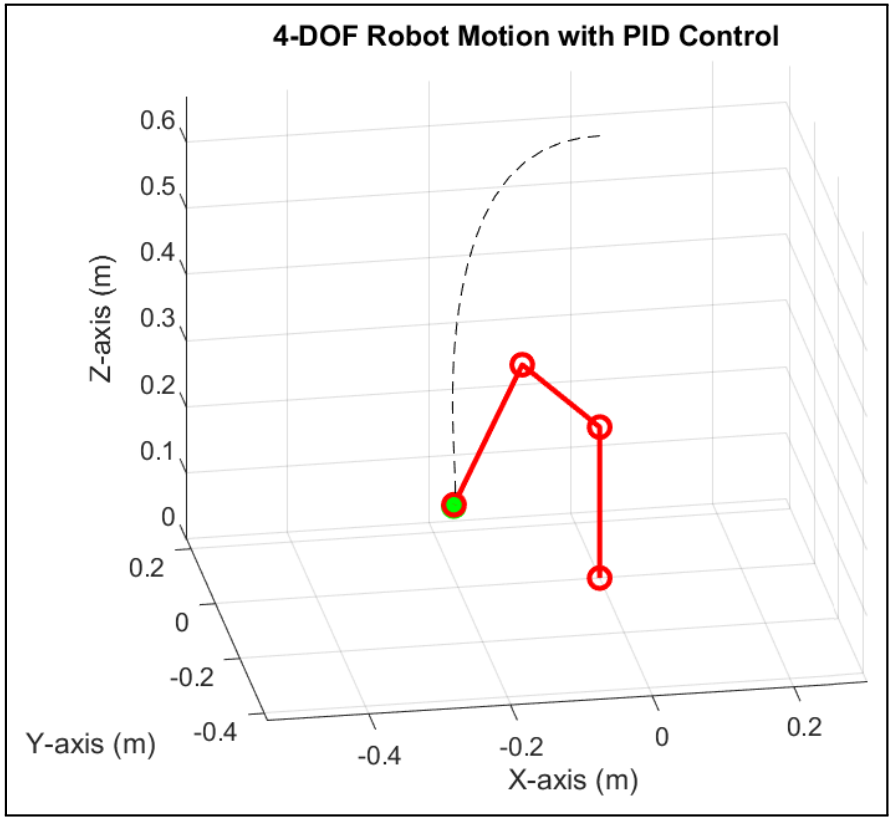
Singularity-Avoidance based Jacobian

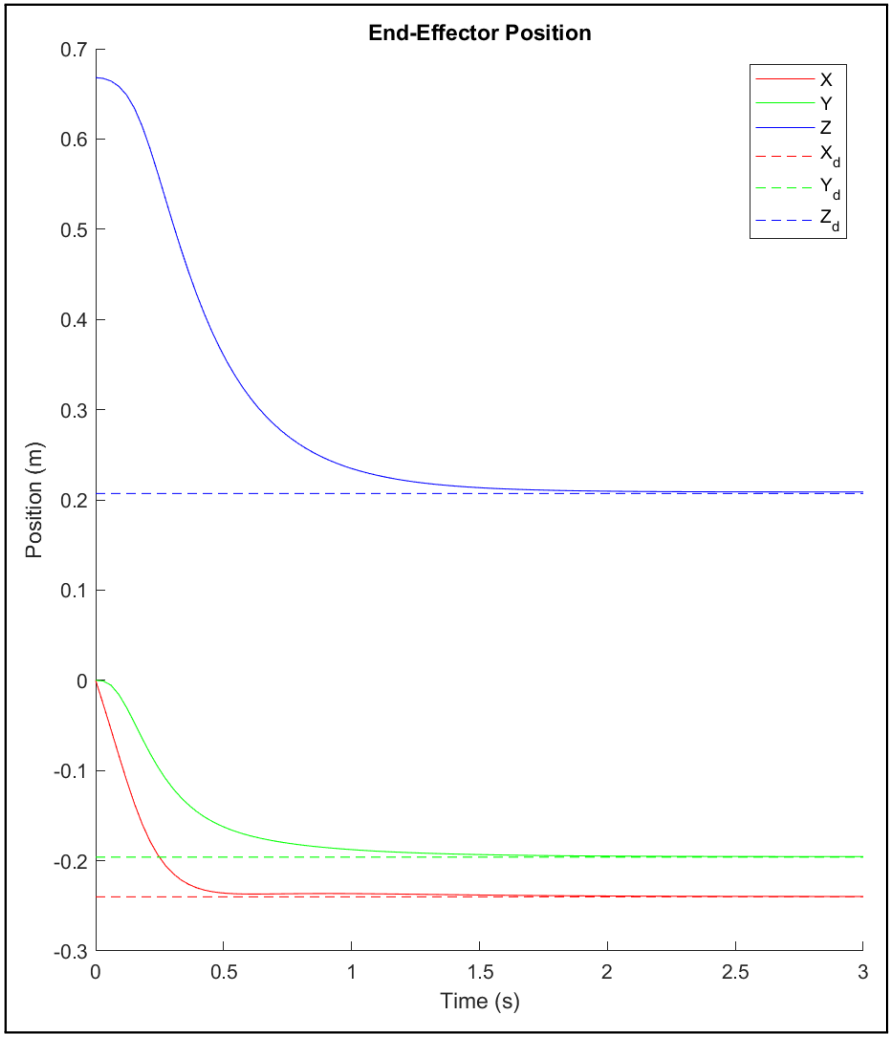
- Although the previous method is useful, at singular configurations (where JJ^T loses rank, i.e., $\det(JJ^T) \approx 0$), the pseudo-inverse J^+ calculation becomes unstable, potentially causing infinite joint velocities.
- Recall: $\dot{q} = J^+ u$, with $J^+ = J^T(JJ^T)^{-1}$.
- To avoid this, we introduce a damping factor λ multiplied by an identity matrix I. This is added to JJ^T before inversion, ensuring the matrix remains invertible. This method is known as the Damped Least Squares (DLS) or Singularity-Avoidance Jacobian:

$$J_\lambda^\dagger = J^T(JJ^T + \lambda^2 I)^{-1}$$

  (Note: The presentation notation $J = J^T(JJ^T + \lambda^2 I)^{-1}$ likely intended to define the damped pseudo-inverse $J_\lambda^\dagger$).
- λ^2 is the damping factor.
- PID controller remains the same as the previous method.

Singularity-Avoidance Jacobian: Simulation Results





Q-Learning based Multi-Agent RL method

- To avoid the issues involved with Jacobian matrices (computation complexity, singularities), we explore Reinforcement Learning (RL) based methods, inspired by the referenced paper.
- The paper proposed a novel actor-critic based Multi-Agent Reinforcement Learning (MARL) method.
- We explore along similar lines but implement a Q-Learning based MARL approach (critic-only aspects mentioned) assuming each joint is an agent.
- Our results are obtained from simulations and may differ from the paper's actor-critic approach.

Q-Learning based MARL: Concept

- Each joint acts as an independent agent aiming to reach a configuration that moves the end-effector to the desired position. Hence, "Multi-Agent".
- The policy learned is online: agents interact directly with the simulated environment, receive rewards, and update their behavior (actions).
- Goal: Learn a policy (π) that minimizes the error $e = x_d - x$ between the desired end-effector position x_d and the current position x.

Q-Learning based MARL: Update Rules

- Q-Learning involves agents maintaining a Q-table (or function approximator) estimating the expected return (cumulative future reward) for taking action a in state s.
- Q-values are updated using the Temporal Difference (TD) error:

$$Q(s,a) \leftarrow Q(s,a) + \alpha_a \times \text{TD_error}$$

  The presentation defines TD error using a value function V(s):

$$\text{TD_error} = R(s,a,s') + \gamma V(s') - V(s)$$

  (Note: This suggests a connection to Actor-Critic or SARSA rather than standard Q-learning's $\max_{a'} Q(s',a')$. The presentation mentions "only critic based" and a "shared V-value".)
- The shared critic V-value is updated as:

$$V(s) \leftarrow V(s) + \alpha_c \times \text{TD_error}$$

  where α_a is actor learning rate (if applicable), α_c is critic learning rate, and γ is the discount factor (0 < 1).

Q-Learning based MARL: Reward Function

- Reward function design is crucial in RL.
- Objective: Reach the desired position x_d while avoiding overly fast or large joint movements, considering real-world mechanical constraints.
- Reward structure penalizes both position error and excessive joint movement:
  - Position Error Component: $r_{\text{pos}} = -\|x_d - x_{\text{new}}\|_2^2$ or $-\|e_{\text{new}}\|_2$ (The presentation uses $-\|e_{\text{new}}\|_2$). Let's use the presentation's form:

    $$\text{position_error_reward} = -\|e_{\text{new}}\|_2$$

    where e_{new} is the error after taking the action.
  - Movement Penalty Component:

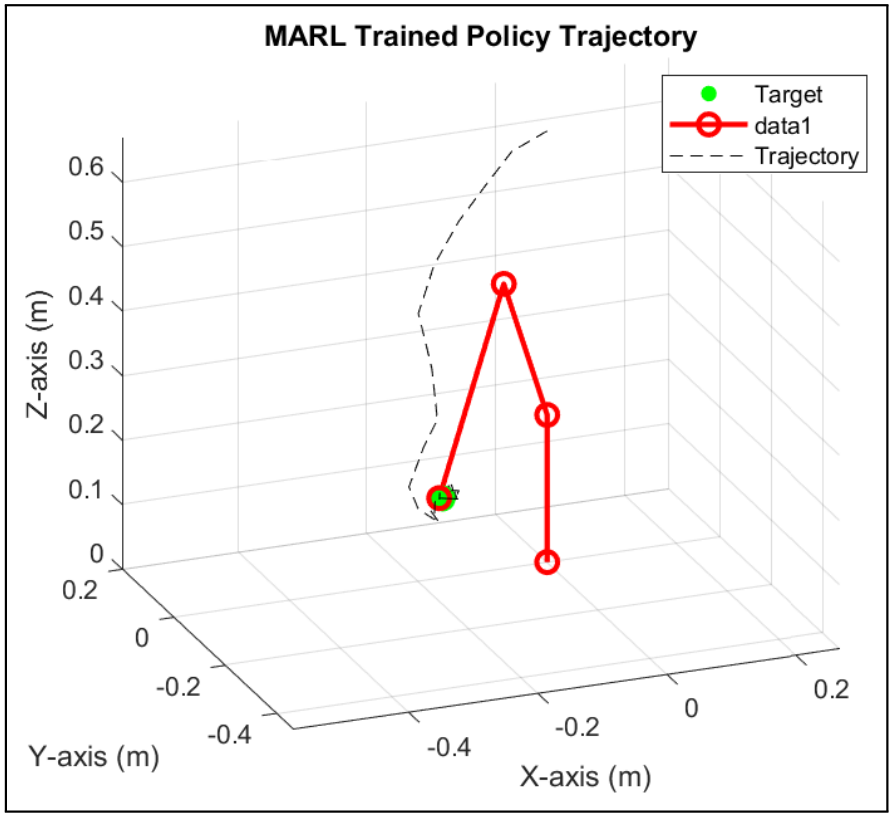
    $$\text{movement_penalty} = -k \times \|\Delta q\|_2^2$$

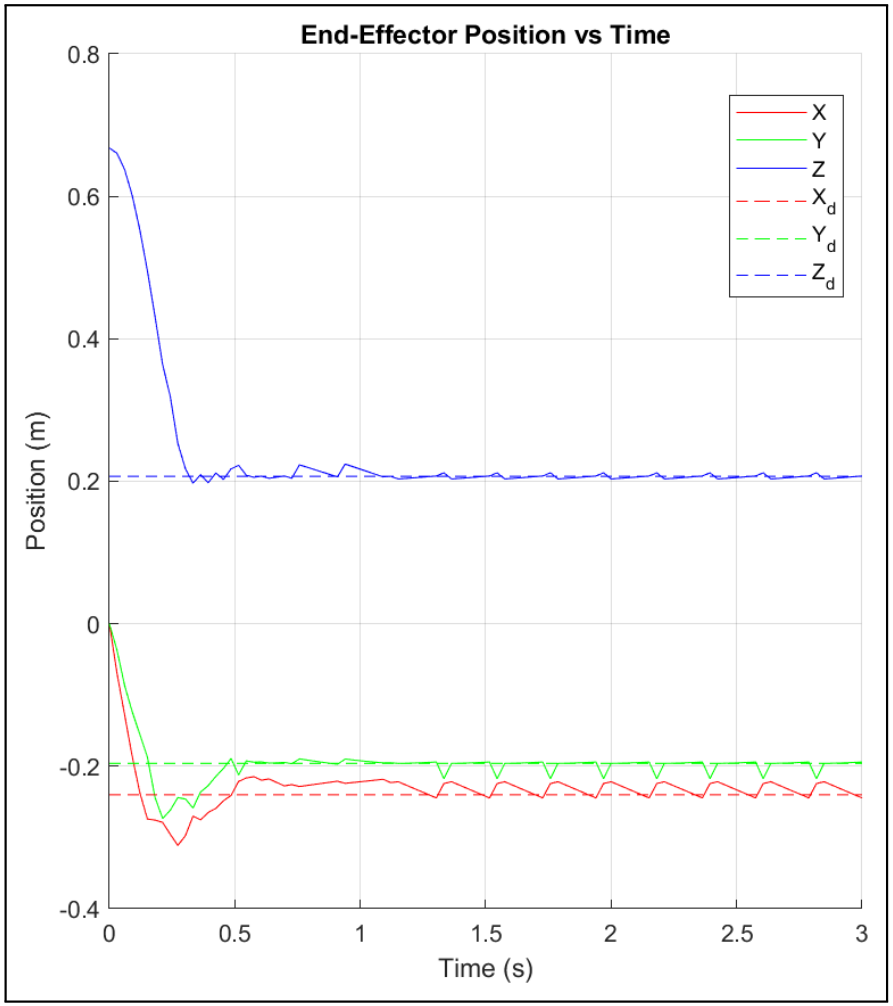
    The presentation uses $k = 0.1$:

    $$\text{movement_penalty} = -0.1 \times \|\Delta q\|_2^2$$

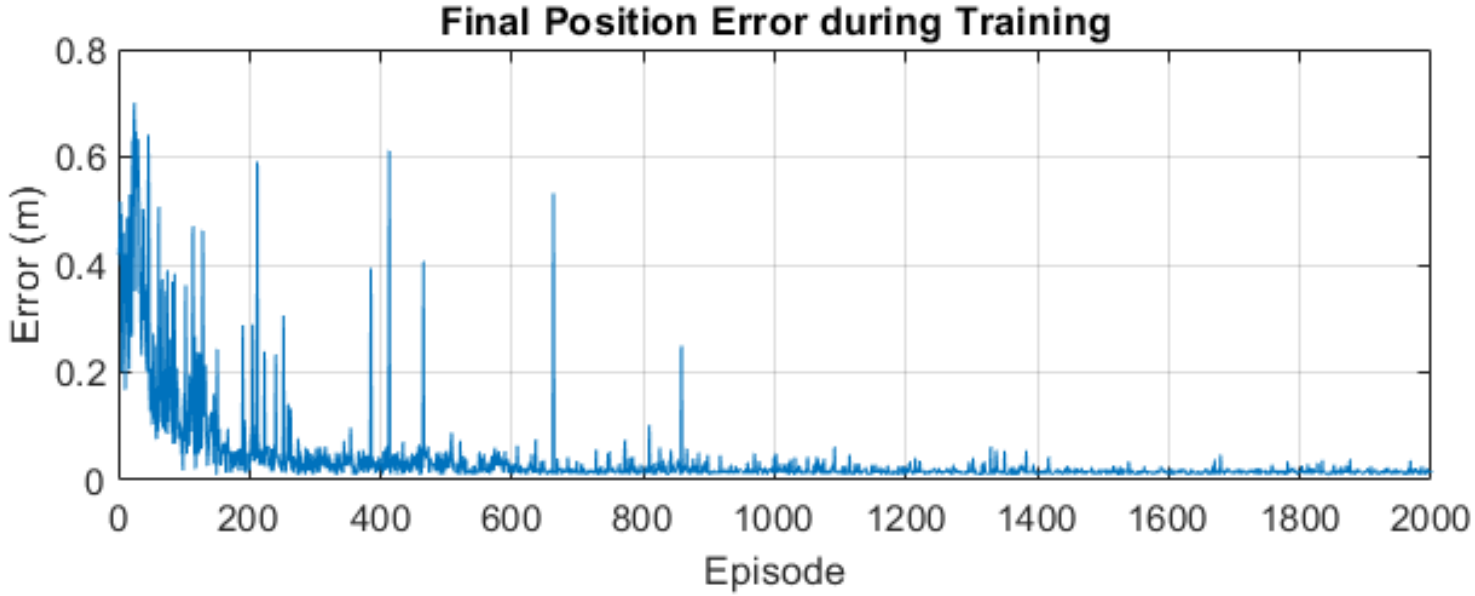
    where Δq is the change in joint angles.
- Total reward R is likely a combination, e.g., $R = w_1 r_{\text{pos}} + w_2 \text{movement_penalty}$.

MARL Simulation Results: Trajectory Tracking





MARL Simulation Results: Training Progress



Conclusion

- Comparing simulation results, the implemented RL-based method appears to achieve faster settling times for trajectory tracking compared to the traditional Jacobian-based methods explored.
- It should be noted that the hyperparameters for the MARL implementation were not extensively tuned, so performance might be further improved.
- While this specific implementation may not be optimal, the results support the potential for MARL methods to outperform traditional control approaches for this type of redundant robot control task, aligning with the intention of the investigation.

Thank You

Kothawale Chetan (M24IRM003)

Vishwanath R (M24IRM010)

Indian Institute of Technology Jodhpur